> aula_14.10

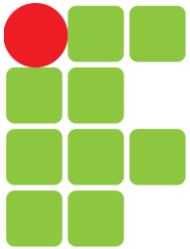
JavaScript

Parte 3 · arrays e métodos

Desenvolvimento de Sistemas Web · 14/10

Prof. Thiago Araújo

IFPE Campus Palmares · ADS 3º período



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO

> mapa_da_aula

Mapa da aula

01

?

Checagem da leitura

02

[ ]

Arrays: o básico

03

map

map e filter

04

Σ

reduce e find

05

. . .

Desestruturação e spread

06

>_

Mão na massa

Hoje a aula é invertida

1. **Vocês já leram**
   o roteiro da MDN no Classroom
2. **Checagem rápida**
   quatro perguntas, sem nota
3. **Eu tiro as dúvidas**
   só o que travou na leitura
4. **Mão na massa**
   a maior parte do tempo

01

Checagem da leitura

Quatro perguntas para ver o que ficou



> checkpoint

O que aparece?

```
const frutas = ['maçã', 'uva', 'caju']
frutas.at(-1)
```

A `'maçã'`

B `'uva'`

C `'caju'`

D `undefined`

> resposta

O que aparece?

```
const frutas = ['maçã', 'uva', 'caju']
frutas.at(-1)
```

A `'maçã'`

B `'uva'`

C `'caju'`

D `undefined`

> checkpoint

Qual método devolve um array novo com cada item transformado?

A `filter`

B `map`

C `find`

D `forEach`

> resposta

Qual método devolve um array novo com cada item transformado?

A `filter`

B `map`

C `find`

D `forEach`

> checkpoint

O que retorna?

```
[5, 12, 8, 20].filter(n => n > 10)
```

A `[12, 20]`

B `[5, 8]`

C `true`

D `12`

> resposta

O que retorna?

```
[5, 12, 8, 20].filter(n => n > 10)
```

A `[12, 20]`

B `[5, 8]`

C `true`

D `12`

> checkpoint

E agora?

```
[5, 12, 8, 20].find(n => n > 10)
```

A `[12, 20]`

B `12`

C `20`

D `true`

> resposta

E agora?

```
[5, 12, 8, 20].find(n => n > 10)
```

A `[12, 20]`

B `12`

C `20`

D `true`

02

Arrays: o básico

Uma lista de valores, em ordem



> conceito

Um array é uma fila numerada




```
const pedido = ['Açaí', 'Coxinha', 'Suco', 'Pastel']
pedido[0]        // 'Açaí'   (começa no zero!)
pedido.length    // 4
pedido.at(-1)    // 'Pastel'
```

O primeiro item mora na posição **0**.

Mexendo no array

+

`push(x)`

adiciona no fim

−

`pop()`

remove do fim

?

`includes(x)`

tem esse item? true ou false

#

`indexOf(x)`

em que posição está? (-1 se não)

> erro_comum

Cuidado com o sort

✗ Evite

```
const n = [10, 9, 100]
n.sort()
// [10, 100, 9]
// e alterou o n!
```

Ordena como **texto** e muda o original.

✓ Prefira

```
const n = [10, 9, 100]
n.toSorted((a, b) => a - b)
// [9, 10, 100]
// n continua intacto
```

Função de comparação e **cópia** nova.

03

map e filter

Transformar e selecionar, sem laço manual



Imperativo × declarativo

`for` Imperativo

```
const dobros = []
for (const n of nums) {
  dobros.push(n * 2)
}
```

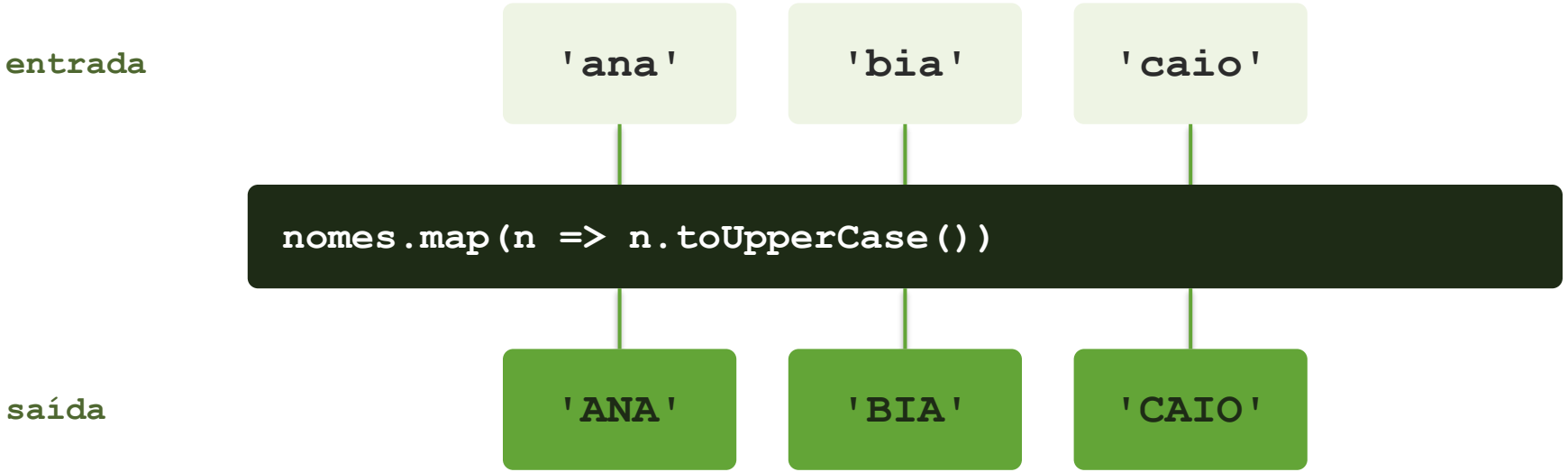
Você diz **como** fazer, passo a passo.

`map` Declarativo

```
const dobros =
  nums.map(n => n * 2)
```

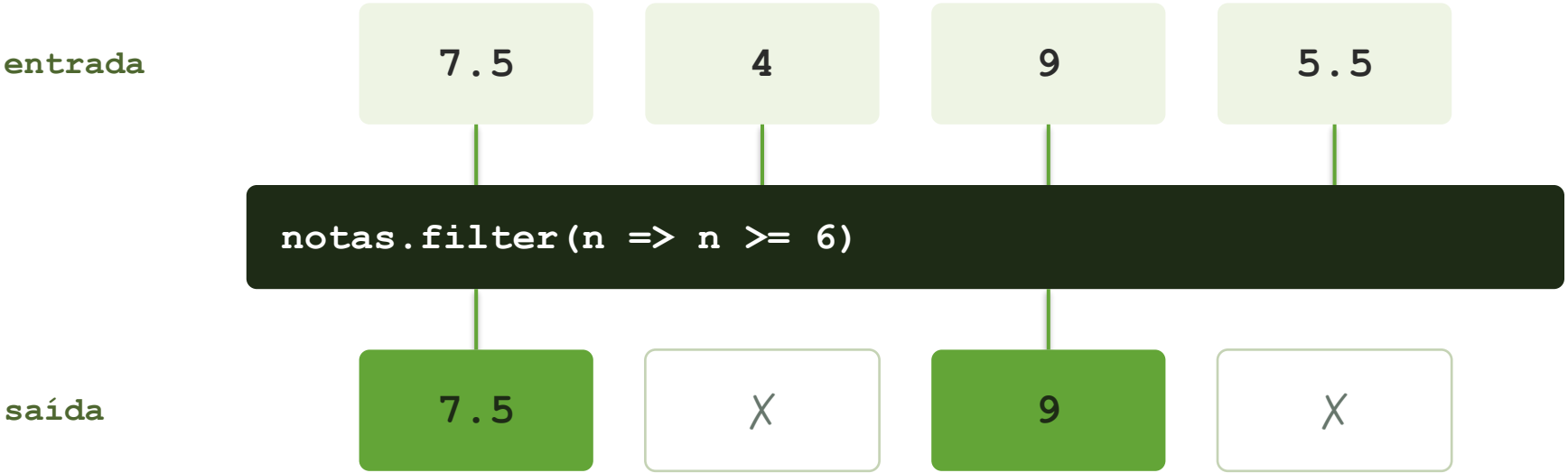
Você diz **o que** quer. O como é do map.

map: transforma cada item



Mesmo tamanho: **um** item de saída para cada item de entrada.

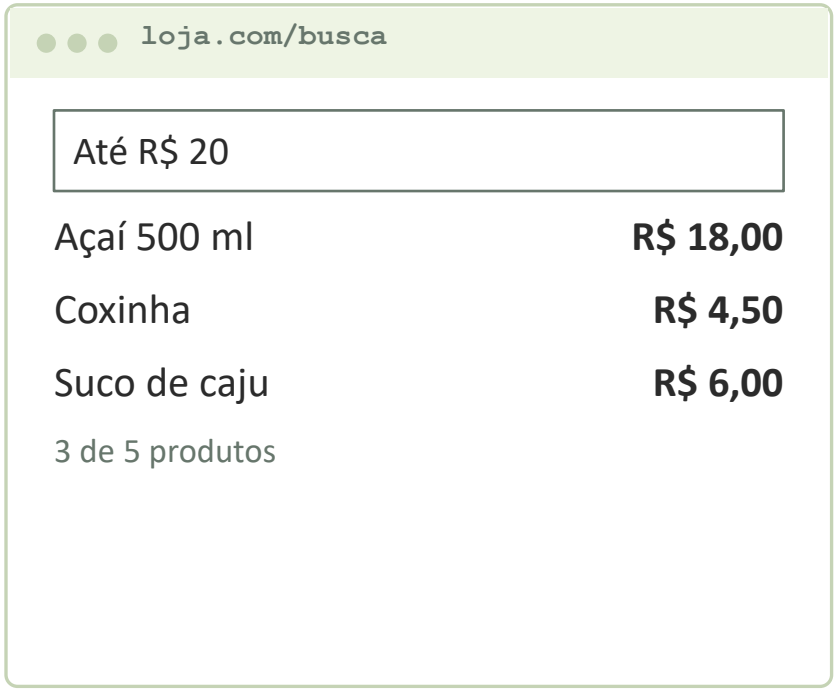
filter: seleciona itens



Só passa quem devolve **true**. A saída pode encolher.

> na_prática

Toda tela de app usa isso



Todo filtro de busca é um **filter**.

Toda lista na tela nasce de um **map**.

```
produtos
  .filter(p => p.preco <= 20)
  .map(p => p.nome)
```

```
[🌽, 🐄, 🐔].map(cozinhar)
  → [🍿, 🍔, 🍳]
[🍿, 🍔, 🍳].filter(ehVegetariano)
  → [🍿, 🍳]
[🍿, 🍳].reduce(comer)
  → 💩
```

Meme clássico que resume a aula inteira.

Versão livre do meme da comunidade JavaScript

> checkpoint

O que retorna?

```
['a', 'b'].map(x => x + x)
```

A `'aabb'`

B `['aa', 'bb']`

C `['a', 'b']`

D `[2, 2]`

> resposta

O que retorna?

```
['a', 'b'].map(x => x + x)
```

A `'aabb'`

B `['aa', 'bb']`

C `['a', 'b']`

D `[2, 2]`

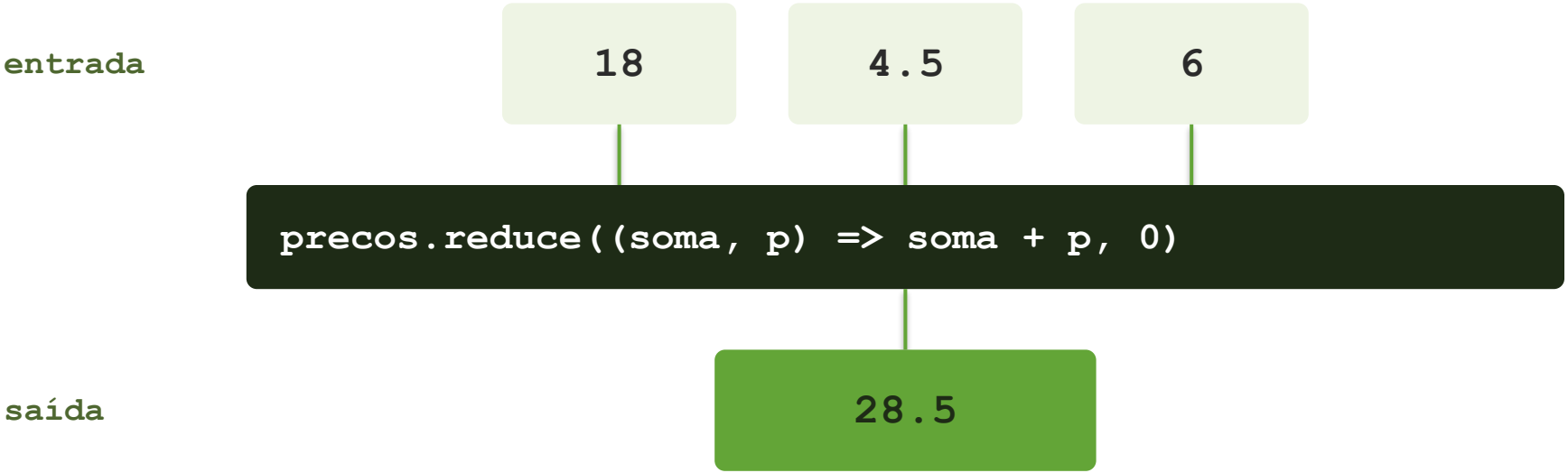
04

reduce e find

Resumir numa resposta e achar o primeiro



reduce: junta tudo em um



Resume o array num **único** valor. O 0 é o início da soma.

reduce, volta a volta



Início

`soma` começa em 0

1ª volta

0 + 18 = 18

2ª volta

18 + 4.5 = 22.5

3ª volta

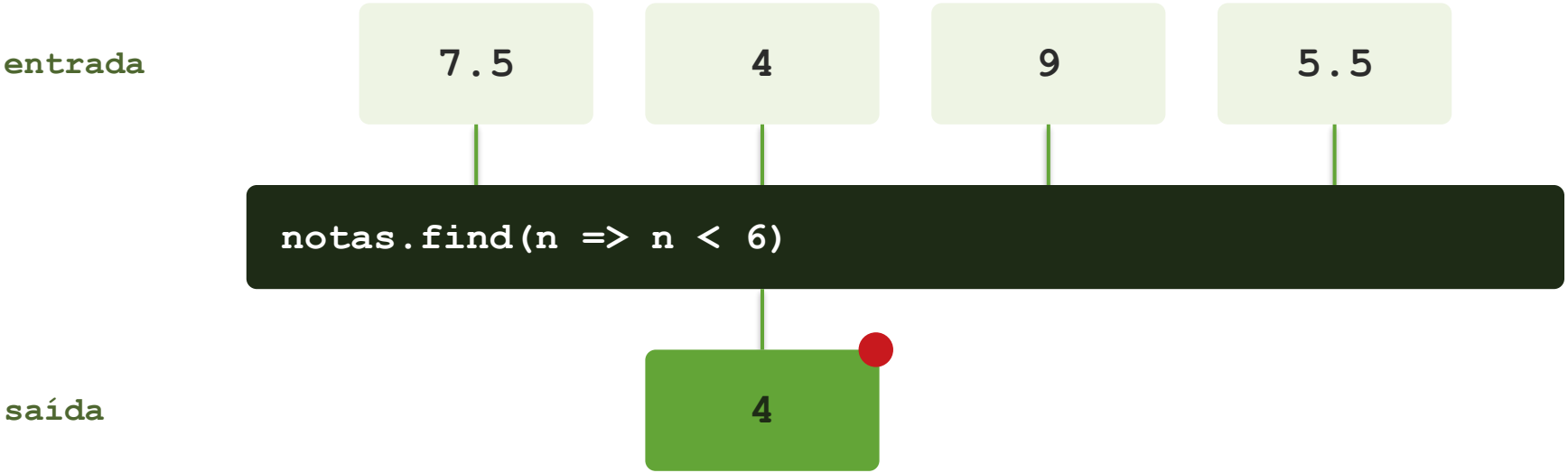
22.5 + 6 = 28.5

Fim

devolve 28.5

```
const total = precos.reduce((soma, p) => soma + p, 0)   // 28.5
```

find: acha o primeiro



Devolve o **primeiro** que passa, ou **`undefined`**.

Qual método usar?

map
transformar cada item

filter
selecionar alguns itens

reduce
resumir em um valor

find
achar o primeiro que serve

- **Pergunte: o que eu quero de volta?**

Os métodos se **encadeiam**: a saída de um é a entrada do próximo.

```
const total = carrinho
  .filter(i => i.disponivel)
  .map(i => i.preco * i.qtd)
  .reduce((s, v) => s + v, 0)
```

Leia de cima para baixo, como uma receita.



> checkpoint

O que retorna?

```
[1, 2, 3, 4]
  .filter(n => n % 2 === 0)
  .map(n => n * 10)
```

A `[10, 20, 30, 40]`

B `[20, 40]`

C `[2, 4]`

D `60`



> resposta

O que retorna?

```
[1, 2, 3, 4]
  .filter(n => n % 2 === 0)
  .map(n => n * 10)
```

A `[10, 20, 30, 40]`

B `[20, 40]`

C `[2, 4]`

D `60`

05

Desestruturação e spread

Tirar da caixa e espalhar



Desestruturar: tirar da caixa

✗ Evite

```
const primeiro = nomes[0]
const segundo = nomes[1]
```

Uma linha por item.

✓ Prefira

```
const [primeiro, segundo]
  = nomes
```

Os colchetes à esquerda pegam **por posição**.



Com objetos, a desestruturação pega pelo **nome**.

Objetos são o tema de 21/10. Aqui, só a primeira mordida.

```
const aluno = {
  nome: 'Ana',
  nota: 9,
  curso: 'ADS'
}

const { nome, nota } = aluno
// nome: 'Ana', nota: 9
```




> erro_comum

Spread: copiar de verdade

✗ Evite

```
const b = a
b.push(3)
// a também mudou!
```

`b = a` não copia: aponta pro **mesmo** array.

✓ Prefira

```
const b = [...a, 3]
// a intacto
// b é um array novo
```

`...` espalha os itens num array **novo**.

Rest faz o contrário: **junta** argumentos soltos num array.

Mesmos três pontinhos: no parâmetro juntam, na chamada espalham.

```
function somar(...nums) {
  return nums.reduce(
    (s, n) => s + n, 0)
}

somar(1, 2, 3)      // 6
somar(5, 10, 15)    // 30
```




> checkpoint

O que aparece no console?

```
const a = [1, 2]
const b = a
b.push(3)
console.log(a)
```

A `[1, 2]`

B `[1, 2, 3]`

C `[3]`

D Erro: `a` é const



> resposta

O que aparece no console?

```
const a = [1, 2]
const b = a
b.push(3)
console.log(a)
```

A `[1, 2]`

B `[1, 2, 3]`

C `[3]`

D Erro: `a` é const

06

Mão na massa

Três desafios, sem nenhum for



> desafio

Notas da turma

⏱ 20 min

1. Calcule a média com **`reduce`**.
2. Maior nota: **`Math.max`** com spread.
3. Quantas acima de 7? **`filter`**.
4. Aprovadas (≥ 6) do maior ao menor.

Regra: nenhum `for`, nenhum `while`.

```
const notas = [
  7.5, 9, 4.5, 8,
  6, 10, 5.5
]

// média: 7.21...
// maior: 10
// acima de 7: 4
// [10, 9, 8, 7.5, 6]
```




> desafio

Cardápio da lanchonete

⏱ 25 min

1. Liste só os nomes (**`map`**).
2. Itens até R$ 10 (**`filter`**).
3. Primeiro acima de R$ 15 (**`find`**).
4. Soma do cardápio todo (**`reduce`**).

```
const cardapio = [
  { nome: 'Coxinha', preco: 5 },
  { nome: 'Pastel', preco: 8 },
  { nome: 'Açaí', preco: 18 },
  { nome: 'Suco', preco: 6 },
  { nome: 'X-tudo', preco: 22 },
]
```

Bônus: nomes dos itens até R$ 10, encadeado.



> desafio

Ranking sem bagunçar

⏱ 20 min

1. Ordene por nota com **`toSorted`**.
2. Pegue os 3 primeiros (**`slice`**).
3. Fique só com os nomes (**`map`**).
4. Prove que **`alunos`** não mudou.

```
const alunos = [
  { nome: 'Ana', nota: 9 },
  { nome: 'Bia', nota: 6.5 },
  { nome: 'Caio', nota: 8 },
  { nome: 'Davi', nota: 10 },
  { nome: 'Eva', nota: 7 },
]
// ['Davi', 'Ana', 'Caio']
```

Entrega: os três desafios no Classroom.

> resumo

Leve para casa

map

map transforma e **filter** seleciona.

Σ

reduce resume e **find** acha o primeiro.

. . .

. . . copia e junta. Prefira não mutar.



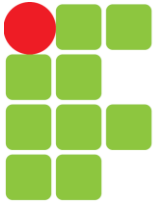
> próxima_aula

Objetos, classes e JSON

21/10 · live coding

Leia: MDN · O básico sobre objetos
Termine os desafios no Classroom

Prof. Thiago Araújo · IFPE Campus Palmares · ADS 3º período



INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PERNAMBUCO